

CSCI 400 – Project Idea Proposal Group #7

1. Project Title

Password Cracking and Strength Evaluation Analysis of the GroceryGo Web Application

2. Objective

The objective of this project is to evaluate and improve the password security mechanisms used in the GroceryGo web application. Specifically, the project will analyze how vulnerable user passwords are to cracking attacks and assess the effectiveness of hashing algorithms, salting techniques, and password complexity policies.

The goal is to simulate real-world password attacks in a controlled lab environment and measure how different security implementations impact resistance to brute-force and dictionary attacks. This project is significant because weak password storage remains one of the most exploited vulnerabilities in modern web applications.

3. Cybersecurity Aspect Addressed

This project focuses on **web application security**, specifically authentication security and password protection mechanisms.

4. Protocol or Mechanism Focus

The project will focus on: Password hashing algorithms (e.g., SHA-256, bcrypt), Salting mechanisms, Password complexity, enforcement of Brute-force and dictionary attack simulations

When users register in GroceryGo, their passwords are stored in the database. Instead of storing plaintext passwords, secure systems apply a cryptographic hash function. **Hashing:** Converts password into fixed-length output. **Salting:** Adds random data before hashing to prevent rainbow table attacks.

bcrypt: Adaptive hashing algorithm designed to resist GPU-based cracking.

The project will compare insecure hashing (e.g., SHA-256 without salt) against secure implementations (bcrypt with salt) to evaluate cracking difficulty.

5. Motivation and Background Research

As the developer of GroceryGo, I want to ensure that the authentication system follows secure design principles. Many real-world breaches (e.g., LinkedIn 2012 breach) occurred due to weak hashing

implementations. Research shows that unsalted hashes are vulnerable to rainbow table attacks. GPU-based cracking tools can break weak passwords within seconds. Bcrypt and adaptive hashing significantly increase cracking time.

6. Problem Identification (Nature of Attack or Vulnerability)

The vulnerability addressed in this project is **offline password cracking after database compromise**.

If an attacker gains access to the GroceryGo user database, they can extract hashed passwords. Then use tools like Hashcat or John the Ripper to perform brute-force or dictionary attacks and recover weak user passwords. The attack succeeds when weak passwords are used, hashing lacks salting and fast hashing algorithms are implemented. Existing solutions recommend salting and adaptive hashing, but many student-level or small-scale web apps still use insecure storage methods. This project evaluates how vulnerable GroceryGo would be under such an attack.

7. Proposed Approach

The project will be implemented in five phases:

Phase 1. Database and Environment Setup : Setting up lab environment and the password database for users

Phase 2. Baseline Implementation: Store passwords using SHA-256 (no salt), extract hashed passwords, attempt cracking using dictionary attack.

Phase 3. Add Salting: Implement random salt per user, repeat cracking test, compare cracking time.

Phase 4. Implement bcrypt: Replace SHA-256 with bcrypt, repeat cracking attempts, measure time complexity.

Phase 5. Password Strength Evaluation: Enforce password complexity rules, Test weak vs strong password resistance, compare success rates.

Tools Used: GroceryGo (React + Firebase backend), SQL Lite Database, Python for hashing simulation, Hashcat or John the Ripper, Linux VM for controlled cracking environment.

Evaluation Metrics: Time required to crack password, Success rate of dictionary attacks, Impact of salting, Computational cost comparison.

8. Expected Outcomes

The expected outcomes include: Demonstration that unsalted SHA-256 hashes are easily cracked, Evidence that salting prevents rainbow table attacks, Proof that bcrypt significantly increases cracking time, Quantitative comparison of password security levels. This project will improve GroceryGo's authentication security and provide a practical demonstration of secure password storage principles. It will also contribute to understanding how small implementation decisions significantly impact web application security.

9. References

- Bonneau, J. (2012). The science of guessing: analyzing an anonymized corpus of 70 million passwords.
- OWASP Foundation. Password Storage Cheat Sheet.
- Provos, N., & Mazieres, D. (1999). A Future-Adaptable Password Scheme (bcrypt).
- NIST Digital Identity Guidelines (SP 800-63B).